Mobile Face Verification & Geotagged Attendance

Panel Notes - How the System Works | Capstone Project

1. Face Verification - "Is this really the employee?"

Question	Simple Answer
Where does recognition happen?	On the SERVER, not the phone. The phone only takes pictures and sends them; the server does all the AI work using a library called face-api.js.
How is an employee enrolled?	HR takes one reference photo of the employee in the admin website. The system converts that face into a list of 128 numbers (a "face fingerprint") and saves it.
What happens at clock-in?	The phone takes a selfie -> the server brightens it if it's too dark -> converts it into the same 128 numbers -> measures how far apart the two fingerprints are.
How is the decision made?	Distance <= 0.45 -> APPROVED. Between 0.45-0.6 -> SENT TO HR FOR REVIEW. Above 0.6 -> REJECTED. These are the standard thresholds recommended by the library itself.
What if someone else uses my account?	If the face doesn't match but the person is standing in the right location, it is not silently rejected - it is FLAGGED for the supervisor/admin to decide. That is exactly the impersonation case the system is designed to catch.

2. Blink Check - "Is this a live person, not a photo?"

The idea in one sentence: a printed photo or a picture on another phone can't blink - so we require one real blink before the camera is even allowed to capture.

How it works, step by step:

1. While the employee looks at the camera, the phone sends small frames to the server a few times per second.
2. For each frame, the server measures how OPEN the eyes are - a standard measurement called the Eye Aspect Ratio (EAR): eye height divided by eye width.
3. The app builds a running average of how open THIS person's eyes normally are (their personal baseline).
4. The moment one frame reads noticeably BELOW that baseline, that's a blink - liveness passed.
5. Only after the blink does the "hold still" capture step begin. A static photo can never get past step 4.

> Why our threshold looks small (5%): textbooks say a blink drops the EAR by about 30%, but that assumes perfect eye tracking. On real phones, the landmark model can't precisely trace a closed eyelid, so real blinks only measured a 5-8% dip in our testing. We calibrated to real device data, not the textbook.

3. Geotagging - "Is the employee actually at work?"

Question	Simple Answer
How is a work area defined?	The admin drops a pin on a map and sets a CIRCLE RADIUS around it. That circle is the geofence.
How do we know they're inside?	The phone sends its GPS coordinates. The server computes the real-world distance between the employee and the pin using the HAVERSINE FORMULA (the standard formula for distance between two points on Earth).
Two conditions must pass	(1) Distance from the pin <= the radius, and (2) the phone's GPS accuracy must be good enough (within 50 meters by default). If GPS is unreliable, the system rejects rather than guesses.
Checked on the phone or the server?	BOTH - the phone checks first for instant feedback, but the server re-checks every submission, so a tampered app cannot bypass it.

About the radius:

- Allowed range in the form: 25 m to 1,000 m (default 120 m).

- Recommended: 50-150 m for an office building.
- Why not smaller than 25 m? Consumer phone GPS is only accurate to about 5-50 m. A tighter circle than the GPS error means legitimate employees get rejected while standing at the front door.
- Why not 1,000 m for an office? At that size, anyone in the whole neighborhood would pass - it is only sensible for large field sites.

4. The Full Clock-In Flow (what the demo shows)

1. Employee opens the app and starts clock-in.
2. BLINK CHECK - proves a live person is present.
3. PHOTO CAPTURED, stamped with time, date, GPS, and a mini-map.
4. Server verifies the FACE MATCHES the enrolled profile.
5. Server verifies the LOCATION IS INSIDE the assigned geofence.
6. Both pass -> attendance recorded. Face mismatch inside the fence -> flagged for human review. Wrong location -> rejected outright.

5. Honest Limitations (state these before the panel asks)

- Photo spoofing is blocked; video spoofing is not. A video of the person blinking, played to the camera, could pass. Full video-based liveness needs on-device machine learning or a paid cloud service (AWS Rekognition - the hook for it exists in the code but was intentionally scoped out).
- Needs internet. Face analysis happens on the server, so no connection = no clock-in.
- GPS spoofing apps aren't detected. The system validates the coordinates the phone reports; detecting fake-GPS tools is a separate hardening step.
- Lighting matters. Very dark selfies can fail to produce a face fingerprint. Images are auto-brightened to reduce this, and the user is simply asked to retake.
- One reference photo per employee. Major appearance changes (glasses, beard) may push a match into the "HR review" zone - which is by design, since a human then confirms.

6. APIs and Technologies Used

> Important point for the panel: the system uses NO PAID APIs and NO API KEYS. All face recognition runs on our own server - employee photos are never sent to a third-party cloud service.

API / Technology	What it's for	Notes for the panel
Our own REST API (NestJS backend)	All the logic: live face + blink frame detection, attendance submission, geofence validation, leave management	This is the system we built - the phone and admin website are just clients of it
face-api.js (open-source library)	Face detection, eye landmarks, and the 128-number face fingerprint	Runs on OUR SERVER with pre-trained models (TinyFaceDetector, 68-point landmarks, FaceRecognitionNet). A library we host, not an external service - photos stay in our database
Expo Camera	Accesses the phone's front camera for the selfie and blink frames	Built-in device API, free
Expo Location	Gets GPS coordinates + accuracy, and converts coordinates into a readable address (reverse geocoding) for the photo stamp	Uses the phone's own GPS - no Google Maps API needed
OpenStreetMap + Leaflet	The interactive maps where admins draw geofence circles (admin website and supervisor screens)	Free, community-run map data - no API key
Carto basemap tiles (free tier)	The mini-map baked into the captured attendance photo	Free, no-key map images; used because OpenStreetMap's own tile server blocks direct mobile-app traffic

API / Technology	What it's for	Notes for the panel
AWS Rekognition	NOT USED - only a placeholder hook exists in the code	Honest answer if asked: cloud-grade video liveness was scoped out; the connection point is ready as future work

> "Face recognition is done by face-api.js running on our own server, GPS comes from the phone via Expo Location, and maps come from free OpenStreetMap/Carto tiles - so the entire system runs with zero API costs and employee biometric data never leaves our server."